

REST API Constraints

Not repeating the usual stuff this time.

There are a number of constraints, a sort of API design “best practices”, associated with REST APIs:

1. **Client Server architecture:** These are applications/systems which can be scaled and upgraded, and even replaced entirely, mutually independently.
2. **Stateless architecture:** Keeping individual request-response pairs separate from one another helps realize scalability and flexibility of APIs.
3. **Cacheable:** With this, we know what and when to return, instead of recreating/recalculating. Thus saving upon resources and makes APIs highly performant and client experience way better.
4. **Layered approach:** Now we may easily add or remove individual concerns, such as DNS, caching, logging, Audit, Authn, Authz.
5. **Code on Demand:** It allows for flexibility, upgradeability, extensibility.
6. **Uniform Interface:** Just change the HTTP method and get the respective action performed for a specific URL. Furthermore, path variables and request parameters make this even more versatile.

Feel free to Reach out for Feedback, Queries, Collaboration and More:

- **LinkedIn Messaging:** <https://www.linkedin.com/in/rishirajopenminds>
- **GitHub:** <https://github.com/rishiraj88>